

Theory and Methodology

On a pattern sequencing problem to minimize the maximum number of open stacks

Horacio Hideki Yanasse *

*National Space Research Institute, Associate Laboratory for Computing and Applied Mathematics, Postal Box 515 – INPE / LAC,
Postal Code 12201-970 – São José dos Campos, S.P., Brazil*

Received 12 October 1994; accepted 9 November 1995

Abstract

Consider a wood cutting setting where different panels have to be cut from large boards. Each cut panel size is put into a stack which remains opened until the last panel of that size is cut. The problem considered here deals with the sequencing of the patterns in order to minimize the maximum number of open stacks during the production run. A branch and bound method for solving this problem is presented which uses a greedy type scheme to select the next node to branch. © 1997 Elsevier Science B.V.

Keywords: Cutting; Pattern sequencing; Pattern allocation; Tool switches; Flexible manufacturing systems

1. Introduction

Cutting and packing problems have attracted the attention of many researchers [1] for many years due to the challenging difficulties one is faced with when trying to solve them in an efficient manner. Many different combinatorial problems arise in the cutting and packing context, depending upon the particular constraints that must be considered in practical settings. In this work we focus our attention in one of these problems, the so called minimization of open stack problem (MOSP).

The MOSP appears, for instance, in the context of cutting wood where there is a limited storage space around the saw machine and stacks of panels being cut are to be removed only after their orders are

completed. If the number of stacks increases too much, some of them have to be temporarily removed to another storage place and brought back afterwards to have their order completed.

Perhaps, the most important application of the MOSP arises in some production systems. Consider a multi-item production system where the production of an item is triggered by a job. Client's orders are composed of several different item types and the production manager wants to determine a sequence of the jobs in order to keep the number of unfinished clients' orders as low as possible. Unfinished clients' orders implies in higher inventory costs, material handling problems, delayed delivery of the clients goods and consequently, delayed incoming payments, higher transportation costs due to split delivery of the goods and so on. The study of the MOSP is, hence, of great interest in this context.

In this paper we will be referring to the MOSP in the context of wood cutting, therefore, we will be

* E-mail: horacio@lac.inpe.br.

Table 1

Patterns	Panels
1	a a a b d d
2	b d e e f
3	c c d d d d
4	a c c e
5	a a f f
6	e e e f f

sequencing cutting patterns. Patterns may contain different panel types displayed according to some specified layout and a panel type may occur repeatedly in a pattern. Several cutting stages may be necessary to cut a pattern completely and panels of a same type may be obtained in different cutting stages. Also, in order to satisfy a client's demands, it is often the case that some cutting patterns must be cut more than once.

We assume that a stack is opened for every new panel type cut and it remains opened until the last piece of that panel type is cut. We also assume that a completed stack can be removed only after a pattern is completely cut, hence, the maximum number of stacks occurs just after some pattern is cut and before any completed part stack is removed. If we consider, for instance, the set of patterns given in Table 1, the sequence of patterns 1, 2, 3, 4, 5, 6 produces, respectively, the following open stacks: (a, b, d), (a, b, d, e, f), (a, c, d, e, f), (a, c, e, f), (a, e, f) and, finally, (e, f). The maximum number of stacks (5) occurs just after cutting pattern 2, before removing the stack corresponding to panel type b which was completed at this stage. The maximum number of stacks also occurs after cutting pattern 3, before removing the stack corresponding to panel type d which was completed at this stage.

Observe that for this small set of 6 patterns problem there are already $6! = 720$ possible different sequences to check. The number of possible different sequences grows exponentially with the number of patterns in the problem. Among all the possible sequences, we are interested in finding one that minimizes the maximum number of open stacks. Among all possible sequences, perhaps only a few produce this minimum, and finding one of them seems to be a quite difficult task.

To minimize the number open stacks one may try

to adopt the policy of maintaining a stack open as little as possible and, consequently, achieving the desired objective. In other words, we may try to minimize the "lifetime" of an open stack, called its spread. The minimization of stack (order/panel) spread is called pattern allocation or cutting sequencing [6]. The pattern allocation or cutting sequencing problem appears in many industrial settings and it may be a quite important issue in the production process. For instance, in the glass industry [6] there might be problems with the identification of parts, matching materials, follow up administration etc.

Solving the minimization of order spread problem (MORP) however, does not lead to an optimal solution to the MOSP and vice versa. MOSP and MORP are not equivalent although they seem to be very closely related. In fact it may be true that:

Conjecture 1. *There is always a solution that is simultaneously optimal for MOSP and MORP.*

Even if Conjecture 1 is true, the result will not be of much help for MORP is itself an NP-hard problem [2,3].

Observe that MORP is a special case of bandwidth minimization [2], and it is equivalent to an important problem that arises in connection with matrices which is the one of finding a permutation of rows and columns of the matrix in order to make it as diagonal as possible.

If we want to minimize the number of open stacks, another possible idea that could come out is to minimize the discontinuities of the panel types being cut, that is, once the first piece of a panel type is cut (a stack is open), try to continue, without interruptions, the cutting of that panel type until its demand is fully completed (the stack is completed). In other words, try to sequence, one after another, the patterns that contain the same panel type. The minimization of discontinuities problem (MDP) reduces to a travelling salesman problem with special characteristics. The MDP was used to solve the MORP [4,6] but, we must point out that these problems are not equivalent. An optimal solution to MORP not necessarily is optimal to MDP and vice-versa. MDP is also not equivalent to MOSP although, again, they seem to be quite closely related.

In fact, the following statements might also be true:

Conjecture 2. *There is always a solution that is simultaneously optimal for MOSP and MDP.*

Conjecture 3. *There is always a solution that is simultaneously optimal for MORP and MDP.*

MORP and the travelling salesman problem are NP-hard. MOSP is probably an NP-hard problem too, but this is still an open question. However, in case it is solvable by a polynomial algorithm, it might be of some use in determining an optimal solution for the MORP and the MDP, in case Conjectures 1 and 2 are true.

A heuristic method to solve the MOSP was proposed by Lins [5] for the particular case where there are at most 2 different panel types per pattern and the graph obtained associating the nodes to panels and arcs to patterns is an “almost” tree, i.e. has exactly one polygon [5] (a polygon is a connected nonempty subgraph with valency 2 in all vertices). The problem in this case reduces to that of traversing trees. Here we deal with general instances of the problem. We are unaware of other works where exactly this same problem is considered.

Similar to other works [4–6], where a related but different problem was considered, it is assumed that the cutting patterns are fixed and the problem is how to sequence them so as to minimize the maximum number of incomplete stacks during a production run. In other words, we are assuming a two stage approach to the original problem and we are considering only the second stage; the first stage being the determination of the cutting patterns.

Observe that in the wood cutting setting, better overall solutions would certainly be obtained if we could solve the cutting problem as a whole, the pattern generation problem together with the sequencing problem. This however is a very difficult task and it is not the issue considered in this work. Other practical approaches that avoid the undesired increase of open stacks can be suggested, for instance, by limiting the cutting patterns so that they contain only a few distinct panel types [8].

A mathematical formulation of MOSP is presented in the next section. It is based on the mathe-

matical model of the minimization of tool switches problem in a flexible manufacturing system [7]. A branch and bound algorithm for the problem is proposed in Section 3, which uses a greedy approach to select the next node to branch. In the final section, some additional remarks are presented.

2. Mathematical formulation of the MOSP

Besides the MORP and the MDP, the open stack problem is also closely related with the problem of minimizing the number of tool switches (MTSP) in a flexible manufacturing machine [7].

The MTSP arises in the following context. Assume that each of N jobs must be processed on a machine that has positions for C tools, that is the tool magazine has capacity C . No job requires more than C tools and if the required tools are not in the machine, then one or more tool switches must occur before the job can be processed. The problem consists of finding the sequence in which to process the jobs and tools to place on the machine before each job is processed in order to minimize the total number of tool switches. A tool switch occurs when a tool is removed from the machine and a different tool is inserted on the machine.

Tang and Denardo [7] present a mathematical formulation for the MTSP and we show next how to model the MOSP based on their formulation.

For ease of explanation, we reproduce Tang and Denardo's formulation. Let

- N be the number of jobs to be processed;
- M be the total number of tools;
- A_j an M -vector of 1's and 0's giving the tool requirements for job j (1 if tool is required, 0 otherwise);
- time n the moment in time after processing the n th job, but before any tools are switched;
- C capacity of the tool magazine;
- e $1 \times M$ vector of 1's;
- x_{jn} = 1 if job j is the n th job in the sequence, 0 otherwise;
- W_n an M -vector of 1's and 0's giving the tools in machine at time n ;
- P_n an M -vector that shows the tool switches that occur at instant n ;

The minimization of tool switches problem (P) can, hence, be formulated as [7]:

$$v = \text{Minimize } \sum_{n=1}^{N-1} eP_n$$

Subject to

$$P_n \geq W_{n+1} - W_n \quad n = 1, \dots, N-1, \quad (1)$$

$$P_n \geq 0 \quad n = 1, \dots, N-1, \quad (2)$$

$$e \cdot W_n \leq C \quad n = 1, \dots, N-1, \quad (3)$$

$$x_{jn} A_j \leq W_n \quad j = 1, \dots, N; n = 1, \dots, N, \quad (4)$$

$$\sum_{n=1}^N x_{jn} = 1 \quad j = 1, \dots, N, \quad (5)$$

$$\sum_{j=1}^N x_{jn} = 1 \quad n = 1, \dots, N, \quad (6)$$

$$x_{jn} \in \{0, 1\} \quad j = 1, \dots, N; n = 1, \dots, N, \quad (7)$$

$$W_n \quad n = 1, \dots, N, \text{ is a vector of 0's and 1's.} \quad (8)$$

There are C tool slots in the machine, hence, it is advantageous to insert C tools on the machine at the beginning (instant 0), switching tools only if necessary at each instant n ($n = 1, \dots, N-1$). Therefore, in constraint Eq. (3) the inequality sign can be replaced with an equality sign.

Observe that if $C = M$, then there is no need for tool switches. If $C < M$, then a lower bound on the number of tool switches is $M - C$, since each one of the tools have to be used at least once. We denote this lower bound LB_4 .

Let C^* be the optimal solution value of the corresponding MOSP, that is the problem obtained considering the jobs as patterns and the tools as panel types. Then,

Proposition 1. LB_4 is the optimal value of problem (P) if $C \geq C^*$.

Proof. By construction. Use the optimal sequencing solution of the MOSP. The C slots of the tool magazine are filled initially with the first C tools determined with the optimal MOSP sequence. $\square$

So, by Proposition 1, if C^* is known, the optimal value v of problem (P) is also known when $C \geq C^*$.

Also, problem (P) with $C = C^*$ can be used to find a solution to the minimization of the open stack problem. However, C^* is usually not known. What should we do in this case?

The following proposition will allow us to formulate a model for the MOSP:

Proposition 2. The optimal value of problem (P) is strictly greater than the lower bound LB_4 when $C < C^*$, that is, $v > LB_4$ if $C < C^*$.

Proof. Any sequencing of the patterns (jobs) produces at least C^* stacks. This means that if we have only C slots, $C < C^*$, at least one switch has to be made to take a tool out from the magazine which will have to be brought back again afterwards to the tool magazine. Hence, we must have necessarily more than $M - C$ switches. $\square$

Based on Propositions 1 and 2 we can formulate the MOSP as:

Minimize C

Subject to

$$\sum_{n=1}^{N-1} eP_n = M - C$$

and constraints (1)–(8)

Tang and Denardo [7] report disappointing computational results with their formulation, using standard commercial Mathematical Programming codes. We should expect, hence, a similar behaviour with the previous formulation if we follow the same approach. This is a drawback to the scheme of finding C^* by solving a set of minimization of tool switches problems.

Tang and Denardo [7] argue in their paper that their MTSP is NP-hard. However, their argument has flaws. We cannot guarantee that MTSP is NP-hard and even if that is true, we cannot guarantee that MOSP is NP-hard. In fact, MOSP is equivalent to MTSP only when $C = C^*$.

We make the observation that if $C > C^*$, an optimal solution to MTSP may not be an optimal solution to the MOSP. The converse, however, is always true.

When $C < C^*$, an optimal solution to the MTSP may not be an optimal solution to the MOSP and vice-versa.

The examination of the relationship between MTSP and MOSP problems lead us to another conjecture:

Conjecture 4. *There is always a solution that is simultaneously optimal for both problems, MOSP and MTSP.*

We must point out that it is possible to devise a branch and bound method to solve the MOSP exploring, perhaps, some special structure of the mathematical model proposed. Also, upper and lower bounds for the MOSP can be easily obtained. We do not explore further this approach here. Instead, we propose another branch and bound approach which uses a greedy criterion to select the next node to branch.

3. A branching and bound approach

Consider any optimal sequencing of the patterns. Following this sequence, one can establish, in order, the stacks that are being opened and the exact sequence in which they are being completed.

Observe that in order to complete a stack corresponding to some panel type, say j , all the patterns that contain panel type j must be cut. So, in the optimal sequence, all the patterns that contain panel type j (and maybe some others) must appear in the optimal subsequence (obtained from the optimal sequence) starting from the beginning of the optimal sequence and finishing up in the instant when the stack corresponding to panel type j is completed.

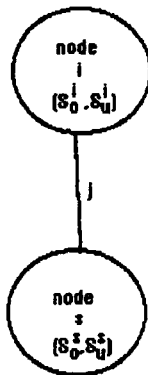


Fig. 1. Identification of a node in the branching scheme.

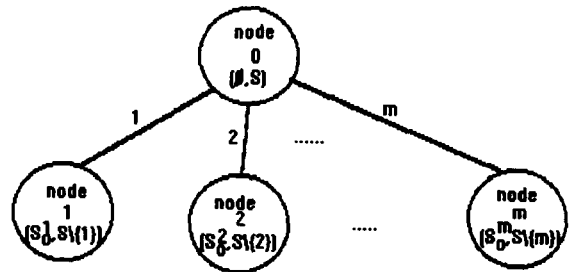


Fig. 2. Branching from node 0.

One can construct a sequence containing the patterns (and only them) that are necessary to complete the stacks following the same order in which they were closed in the optimal sequence. This sequence is also optimal since any of its subsequences up to the instants where a stack is completed contains only a subset of the patterns that is present in the corresponding subsequences of the optimal sequence and, hence, the maximum number of open stacks obtained with this constructed sequence cannot be greater than the one obtained with the optimal sequence, since the order in which the stacks are completed using any one of the sequences is the same.

Therefore, to find an optimal solution to MOSP, it is sufficient to search for an optimal sequencing of the stacks being completed. Our proposed branch and bound scheme explores this observation.

Let S be the set of all part types of the open stack problem, i.e., $S = \{1, 2, \dots, m\}$. Let S_o and S_u be subsets of S such that $S_o \subseteq S_u \subseteq S$.

In our branching scheme, each node will be identified by the set S_u , the set of *unfinished* part types, and the set S_o , the set of currently *open* stacks (see Fig. 1).

For each $j \in S_u^i$, create a new node, say node s , with $S_u^s = S_u^i \setminus \{j\}$ and S_o^s is the set of open unfinished stacks that is obtained if all patterns (and only them) necessary to finish all parts in $\overline{S_u^s}$ ($\overline{S_u^s} = S \setminus S_u^s$) are sequenced.

We start our enumeration with node 0, which has the following starting sets: $S_o^0 = \emptyset$, $S_u^0 = S = \{1, 2, \dots, m\}$. In Fig. 2 we illustrate the branching at node 0.

Observe that with the enumeration proposed, each node has a single predecessor with the exception of node 0, the root node and $|S_u|$ successors where $| \cdot |$

denotes the cardinality of the set. A path from the root node to a leaf of the enumeration tree provides a possible solution to the open stack problem.

A trivial lower bound for the minimization of open stacks problem is the number of parts in each pattern. Let us denote this lower bound LB_1 .

Other lower bounds for the open stack problem can be obtained as follows. Consider any node j in the enumeration scheme. Let v_j^k denote the number of open stacks that results when going from node j to node k , a successor of node j , which was determined by finishing part $t \in S_v^j$ ($S_v^k = S_v^j \setminus \{t\}$; $v_j^k \geq |S_v^j|$; $v_j^k > |S_v^k|$). Then,

Proposition 3. A lower bound for the number of open stacks on node j is the minimum of all v_j^k 's where k is a successor of node j .

Proof. Any path that passes through node j will have to pass through one of its successors. $\square$

We denote the lower bound calculated according to Proposition 3 by LB_2 .

Another lower bound for the number of open stacks on node j is the number of open stacks that resulted when branching from the predecessor of j . If p is the predecessor of j then this lower bound is v_p^j . This lower bound will be denoted by LB_3 .

The overall lower bound for the number of open stacks on node j is the best of all these lower bounds, including the lower bound of its predecessor node, since any node has a single predecessor except

the root node 0. If we denote LB^j this overall lower bound on node j , then:

$$LB^j = \max\{LB_2, LB_3, LB^p\} \quad j > 0,$$

where p is the predecessor of node j , and

$$LB^0 = \max\{LB_1, LB_2\}.$$

We can also determine a trivial upper bound on the number of open stacks at each node which is the total number of parts of the problem still left to be cut. Let us denote this upper bound calculated at a node j by UP^j . Hence, $UP^j = |S_v^j|$.

Let v^{up} be the value of the best solution so far for the open stack problem. Initially $v^{up} = |S|$.

We adopt a depth first search, branching the most promising node, that is, we use a greedy criterion. We choose to branch among the highest level nodes the one that results in the least possible number of incomplete panel types (stacks).

Fathoming of the nodes is performed by comparing the lower and upper bounds obtained at each node with the best solution we have so far. When any lower bound in node j is greater or equal to v^{up} , node j can be fathomed. When $LB^j \geq UP^j$, node j can be fathomed and v^{up} is updated with LB^j if $v^{up} > LB^j$.

The branch and bound scheme is illustrated with the example given in Table 1. We start with node 0, where $S_v^0 = \emptyset$ and $S_u^0 = \{a, b, c, d, e, f\}$. In node 0, $LB_1 = 4$. Hence, $UP^0 = 6$ and $v^{up} = 6$. Branching in node 0 produces 6 new nodes as shown in Fig. 3.

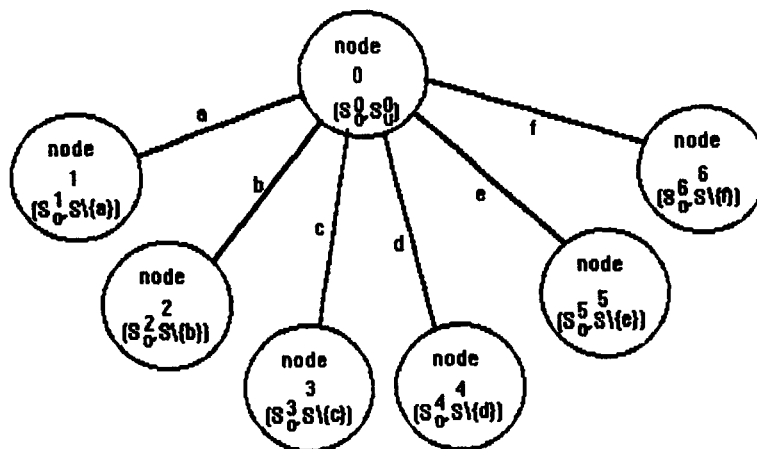


Fig. 3. Branching from node 0.

For these nodes we have:

$$\begin{aligned} S_0^1 &= \{b, c, d, e, f\}, & UP^1 &= 5, & v_0^1 &= 6, & LB_3 &= 6; \\ S_0^2 &= \{a, d, e, f\}, & UP^2 &= 5, & v_0^2 &= 5, & LB_3 &= 5; \\ S_0^3 &= \{a, d, e\}, & UP^3 &= 5, & v_0^3 &= 4, & LB_3 &= 4; \\ S_0^4 &= \{a, b, c, e, f\}, & UP^4 &= 5, & v_0^4 &= 6, & LB_3 &= 6; \\ S_0^5 &= \{a, b, c, d, f\}, & UP^5 &= 5, & v_0^5 &= 6, & LB_3 &= 6; \\ S_0^6 &= \{a, b, d, e\}, & UP^6 &= 5, & v_0^6 &= 5, & LB_3 &= 5. \end{aligned}$$

Hence, in node 0, $LB_2 = \min\{v_0^i, i = 1, \dots, 6\} = 4$ and $LB^0 = \max\{LB_1, LB_2\} = 4$.

Following the scheme we fathom nodes 1, 4 and 5 since the lower bound on these nodes equal the currently upper bound for the optimal value of the problem. We then branch the most promising node which is node 3 (see Fig. 4).

For these nodes we have:

$$\begin{aligned} S_0^7 &= \{b, d, e, f\}, & v_0^7 &= 5; \\ S_0^8 &= \{a, e, f\}, & v_0^8 &= 5; \\ S_0^9 &= \{a, e, f\}, & v_0^9 &= 5; \\ S_0^{10} &= \{a, b, d, f\}, & v_0^{10} &= 5; \\ S_0^{11} &= \{a, b, d\}, & v_0^{11} &= 5. \end{aligned}$$

Hence, in node 3, $LB_2 = \min\{v_0^i, i = 1, \dots, 6\} = 5$ and $LB^3 = \max\{LB_2, LB_3, LB^0\} = 5$. Also, $UP^3 = 5$. Since $LB^3 = UP^3$, any solution “through” this node has an optimal value of 5. We, therefore, update v^{up} to 5 and fathom node 3 (and consequently, all its

successors). Nodes 2 and 6 are also fathomed since their lower bounds equal the currently upper bound. The procedure then terminates with an optimal value of 5 and a solution given by the following: sequence the patterns (and only those) to complete part *c* first, then sequence the patterns (and only those) to complete any other uncompleted part in turns, until all parts are completed.

As can be seen, the branch and bound scheme is quite simple to apply. For the particular example presented, the number of nodes produced with the branching was not prohibitively large.

4. Some observations concerning the proposed branch and bound scheme

All the computations that are required in the proposed scheme can be done easily using simple matrix manipulations.

Let M be an $n \times m$ matrix of 0's and 1's where n is the number of different patterns and m is the number of different panel types. Without loss of generality, we will refer to the patterns as 1, 2, 3, ..., n and the panel types as 1, 2, ..., m . M is built in such a way that m_{ij} , the i th row j th column element of M , is 1 if in pattern i there is a panel type j , and zero, otherwise.

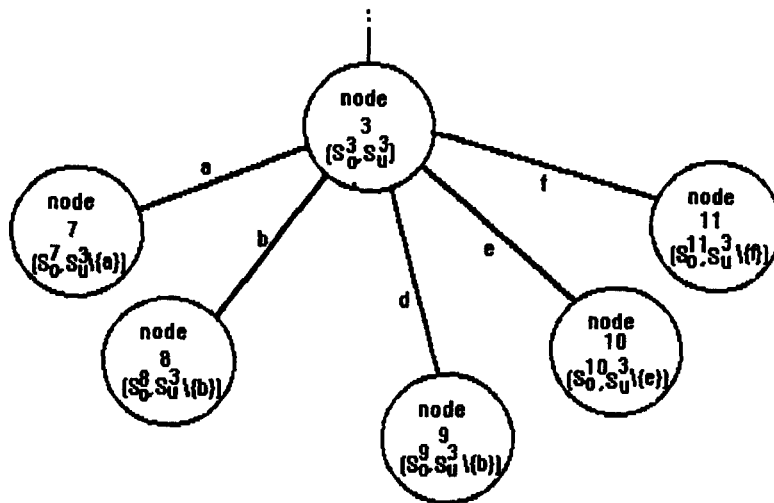


Fig. 4. Branching from node 3.

For the problem data presented in Table 1, the resulting matrix M is

$$M = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

Let W be a vector of size m where w_j , the j th element of vector W , is equal to the number of panel types that will be open if we try to complete all requirements of panel type j . W can be evaluated as indicated next.

Let N_j be a row vector obtained by performing an “or” operation (denoted by $\vee$) with the rows i 's of M , where $m_{ij} = 1$. Each row vector N_j identifies the panel types that will be opened if we decide to complete the panel type j . For instance, for the previous matrix M ,

$$N_1 = [110100] \vee [101010] \vee [100001] \\ = [111111],$$

$$N_2 = [110100] \vee [010111] = [110111],$$

$$N_3 = [001100] \vee [101010] = [101110],$$

$$N_4 = [110100] \vee [010111] \vee [001100] \\ = [111111],$$

$$N_5 = [010111] \vee [101010] \vee [000011] \\ = [111111],$$

$$N_6 = [010111] \vee [100001] \vee [000011] \\ = [110111].$$

So, if we want to complete panel type 3, we observe from N_3 that panel types 1, 3, 4 and 5 will be opened. Observe that $N_i \cdot e'$ gives the number of panel types that will be opened if we want to complete the panel type i , where e is a vector of 1's and e' denotes its transpose.

At some nodes, some panel types may be already opened when we are trying to decide which panel type i to complete. These are considered using a row vector of 0 and 1's, N_{ant} where $n_{ant(i)}$, the i th element of N_{ant} , is 1 if the i th panel type was already opened and it is still incomplete, and 0, otherwise. If we perform $(N_i \vee N_{ant})$, we obtain the

information about the number of opened panel types that is going to result if we try to complete panel type i ; hence, w_i , the number of opened panel types if we try to complete all requirements of panel type i , is given by $(N_i \vee N_{ant}) \cdot e'$.

There might be cases where a tie occurs on the choice of the next node to branch. The breaking of ties can be done by checking whether there are any dominated panel types. By dominated panel types we mean the following:

Let T_i and T_j be the set of patterns (rows in matrix M) that are necessary to be sequenced in order to complete panel types i and j , respectively. We say panel type i is dominated by panel type j if $T_i \subseteq T_j$, which means that if we complete panel type j , we will complete panel type i as well. In case there is a tie and there are dominated panel types, we should choose a node corresponding to the completion of a dominated panel type for the node corresponding to a completion of a dominant panel type will be subsequently considered anyway further on, if worthed, in subsequent branchings. Also, at the current level, the nodes corresponding to the completion of dominant panel types can be fathomed as well.

Consider, for instance, an instance where there are 4 patterns and 5 panel types to complete, the corresponding matrix M is given by

$$M = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

and there is a tie for all the panel types to be completed (due to some stacks already opened). Observe that panel types 2 and 3 (corresponding to columns 2 and 3, respectively in matrix M) are completed if we sequence patterns 1 and 2 (corresponding to rows 1 and 2 of matrix M). Hence, panel type 2 dominates panel type 3 (or vice-versa, in this case). Similarly, panel type 5 dominates panel type 4 since we need to sequence patterns 2, 3 and 4 to complete panel type 5 and sequence patterns 2 and 4 to complete panel type 4.

Following the suggested policy of breaking ties, we should choose for branching the node corresponding to the completion of panel type 2 (or 3) or

panel type 4. If the node corresponding to the completion of panel type 4 is chosen we can fathom in this current level, the node corresponding to the completion of panel type 5 since this case, if worthed, is going to be considered anyway in our scheme when we analyse which node to branch in the subsequent level.

It is possible to improve the search and avoid some duplicate calculation in the branching process by observing that any sequencing of the patterns necessary to finish a set of panel types leads to the same set of open stacks at the end. For instance, in the example, to finish panel types *a* and *c*, we need to sequence patterns 1, 3, 4 and 5. The sequence, 1, 3, 4, 5 or 3, 4, 5, 1 or 3, 4, 1, 5 or 1, 5, 3, 4 or any other possible permutation of these patterns will leave at the end the same open stacks corresponding to panel types *b*, *d*, *e* and *f*. Hence, we may avoid duplicate calculation by carrying an additional set at each node *j*, say set U^j , that contains panel types that need not be considered again for branching.

It can be easily seen that every time a node is chosen to be branched, the sets U^j of each node *j*, at the same level as the chosen branching node, can be updated including in it the panel type corresponding to the node being branched. Consider the example presented previously and recall Fig. 3. $U^0, U^1, U^2, U^3, U^4, U^5, U^6$ corresponding to nodes 0, 1, 2, 3, 4, 5, 6 respectively, are initially empty. Node 3 was chosen to be branched corresponding to finishing panel type *c*. We, hence, can update U^1, U^2, U^4, U^5, U^6 including in them, panel type *c*. This means that if we happen to branch any one of the nodes 1, 2, 4, 5 or 6 we do not need to consider a branch corresponding to finishing panel type *c* since that has been considered elsewhere. Observe that once the set U^j of a node *j* is updated, its corresponding LB_2 needs to be updated.

Carrying the sets U^j in the example presented is not of much help, but it may save some computation time when solving other instances of the problem.

We would like to observe also that the greedy criterion imbedded in the branch and bound scheme proposed can be used directly to generate a feasible solution to the MOSP. This can be viewed as a simple and reasonably fast greedy heuristic. Improvements on the quality of the solution obtained by this heuristic may be achieved by extending

somewhat the search. For instance, using the greedy criterion there will be times when we have to make arbitrary choices, mainly, when we are faced with ties and there are no dominant panel types; we could explore each one of the possibilities and choose the best solution at the end.

Many other heuristics can be proposed to solve this pattern sequencing problem, using other simple strategies: finish eldest open stack next; next pattern in the sequence is the one having the largest number of panel types in common with the current open stacks; next pattern to sequence is the one that results in the least number of open stacks etc. In particular, perturbation schemes like simulated annealing, tabu search, or genetic type algorithms can be easily implemented without much difficulty.

All these simple heuristics can be used to determine upper bounds on each node in our branch and bound scheme. We did not suggest this because the proposed depth first policy and the greedy criterion to select the next node to branch will naturally lead us quickly, to a hopefully good solution (upper bound) to the problem. We believe that the added computational effort to determine upper bounds (which, to be helpful, must be better than the ones we get) at each node using anyone of these heuristics is not overall worthed.

5. Summary

We presented here a branch and bound scheme to find an optimal sequence of cutting patterns that minimizes the maximum number of open stacks during a production run. We are unaware of other works dealing with the general case of the MOSP. This is an initial study where the open stack problem was approached and where an exact method to solve it was proposed.

A few challenging conjectures were raised in this work. We believe that this general sequencing problem is computationally hard to solve. This can be partly substantiated by its relation to the minimization of order spread problem, minimization of discontinuities problem and the minimization of tool switches problem.

The non trivial mathematical formulation presented in this work for the MOSP is based on the model presented by Tang and Denardo [7] for the

minimization of tools switches problem. Recall that they report very disappointing computational results using their model. We would expect a similar computational performance if we had used the mathematical model formulation we proposed for the MOSP since most of the constraints in both models are identical. We chose to develop an alternative solution scheme that searches for a solution exploring some particularities that an optimal solution to the MOSP may have.

Recently, in Yuen [9] some heuristics for pattern sequencing were presented. The objective function in Yuen's paper is however different from the one sought in this manuscript. Hence, there is no point in performing computational test comparisons of the exact procedure presented here and the heuristics presented in his/her paper.

Acknowledgements

Research partially financed by CNPq grant number 522223/94-0. We would like to thank Professor Hisatoshi Suzuki who indicated us reference [7], suggesting that our problem might be related with the tool switches problem, and an anonymous referee for helpful comments that improved the presentation of the paper.

References

- [1] Dyckhoff, H. and Finke, U., *Cutting and packing in production and distribution: typology and bibliography*, Springer Verlag, Heidelberg, 1992.
- [2] Garey, M.R. and Graham, R.L., and Johnson, D.S. and Knuth, D.E., "Complexity results for bandwidth minimization", *SIAM Journal of Applied Mathematics* 34/3 (1978) 477–495.
- [3] Garey, M.R. and Johnson, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, 1979.
- [4] Harris, R., "An industrial application of integer programming", MSc. Thesis in Industrial Programming Technology, University of Sheffield, Sheffield, 1987.
- [5] Lins, S., "Traversing trees and scheduling tasks for duplex corrugator machines", *Pesquisa operacional* 9/1 (1989) 40–54.
- [6] Madsen, O.B.G., "An application of travelling salesman routines to solve pattern allocation problems in the glass industry", *Journal of the Operational Research Society* 39/3 (1988) 249–256.
- [7] Tang, C.S. and Denardo, E.V., "Models arising from a flexible manufacturing machine, Part I: minimization of the number of tool switches", *Operations Research* 36 (1988) 767–777.
- [8] Vasco, F.J. and Wolf, F.E. and Pflugrad, J.A., "An efficient heuristic for planning mother plate requirements at Bethlehem steel", *Interfaces* 21/2 (1991) 1–7.
- [9] Yuen, B.J., "Heuristics for sequencing cutting patterns", *European Journal of Operational Research* 55/2 (1991) 183–190.